RA23E01 – ROBOT OPERATING SYSTEM

AGILEX SCOUT MINI AUTONOMOUS NAVIGATION USING ROS 2

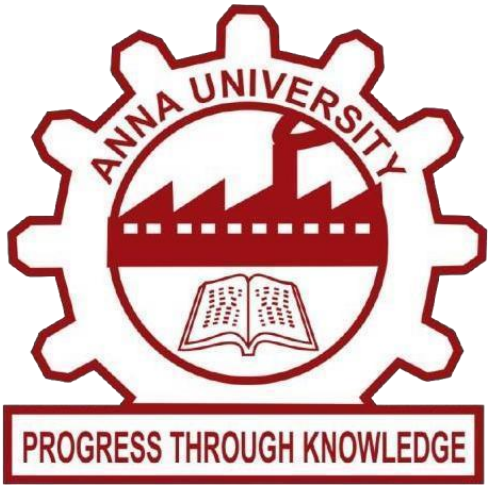
Submitted By:

SAILAKSHMI BT (2024511019)

BACHELOR OF ENGINEERING

in

ROBOTICS AND AUTOMATION



DEPARTMENT OF PRODUCTION TECHNOLOGY
MADRAS INSTITUTE OF TECHNOLOGY
ANNA UNIVERSITY - CHENNAI 600044

BONAFIDE CERTIFICATE

Certified that this project report “AgileX Scout Mini Autonomous Navigation using ROS 2” is the Bonafide work of Sailakshmi BT (2024511019), who carried out the project work under my supervision.

SIGNATURE

SUPERVISOR

TABLE OF CONTENTS:

S.No	TOPICS	PAGE
1	**Introduction**	**5**
2	**Robot Specifications**	**5**
3	**Sensors Used and the Reason of Usage**	**6**
4	**SolidWorks Design**	**6**
5	**Conversion to URDF**	**8**
6	**Importing URDF to ROS**	**9**
7	**Creating the Civil Facility World in Gazebo**	**10**
8	**Creating the Map**	**12**
9	**Teleoperating the Robot Through the Facility**	**13**
10	**Autonomous Navigation**	**14**
11	**Navigation Problems Found and Solved**	**18**
12	**Algorithms Used (Smac 2D, AMCL, EKF, RPP, BT …)**	**19**
13	**System Architecture**	**19**
14	**What Happens When the Code Runs (Flowchart)**	**21**
15	**How to Set It Up**	**22**
16	**Future Enhancements**	**23**
17	**Conclusion**	**23**

Abstract

This report presents the simulation and autonomous navigation of the AgileX Scout Mini four-wheel-drive skid-steer mobile robot using the Robot Operating System 2 (ROS 2 Jazzy) and the Gazebo Harmonic simulator. The robot was modelled in SolidWorks (assembly Assem2), exported to a URDF description, corrected for use in ROS 2 and fitted with a 360-degree 2D LiDAR, an IMU and a front RGB-D camera. A 20 m × 16 m civil facility with six rooms, a central corridor, 1.2 m doorways, columns and furniture was generated as a Gazebo world together with a matching 5 cm occupancy map. The robot is localised by AMCL on this map, with an Extended Kalman Filter that fuses wheel odometry and the IMU, and is driven by Nav2 (Smac 2D A* global planner, rotation-shim and regulated pure pursuit controller).

The mission did not work at first: earlier runs failed at different legs (the robot could not turn, drove into a wall, got wedged in doorways, or lost its position) and each failure had to be diagnosed and fixed. After these fixes, the final recorded run completed a closed-loop mission of 23 Nav2 goals (three rooms and the return to the corridor hub) with no retries. Every arrival was checked against the Gazebo ground-truth pose: the mean error was 0.25 m and the worst 0.33 m against a 0.45 m limit. The report also lists the sixteen navigation problems that were found by running the mission and how each was fixed (wheel friction, footprint, LiDAR self-hits, AMCL noise, goal headings and others), and states the problems that are still open. All results are from simulation; the success is one recorded run after many failed ones, the number of attempts was not logged, and nothing was tested on the physical robot.

1. Introduction

The Robot Operating System (ROS) is an open-source middleware framework for robot software. It does not replace the operating system; it provides the communication layer (nodes that exchange messages on topics, services and actions), a standard set of message types, tools for visualisation and recording, and a large library of ready-made packages for mapping, localisation and navigation. ROS 2 is the current generation and was used here in the Jazzy Jalisco release on Ubuntu 24.04, together with the Gazebo Harmonic physics simulator, the RViz2 visualiser, the robot_localization package for sensor fusion and the Nav2 navigation stack.

Buildings such as laboratories, hospitals and factories increasingly use autonomous mobile robots (AMRs) to carry items or inspect rooms. Such a robot must know where it is, plan a path to a goal and follow it while avoiding walls, furniture and people. Testing these functions directly on hardware is slow, costly and can damage equipment, so the robot is first run in simulation: the robot, its sensors and the building are all models, and the same ROS 2 software that would run on the real robot runs against them.

The robot used in this project is the AgileX Scout Mini, a compact four-wheel-drive skid-steer platform for research and light industrial use. The chassis and wheels were modelled by the author in SolidWorks (assembly Assem2) and converted to a ROS 2 description. The objectives of the project were:

- to convert the SolidWorks model to URDF and make it behave correctly in Gazebo (frames, wheel axes, inertia, friction, drive model);
- to build a Gazebo world that resembles a civil facility, with rooms, doorways, corridor posts and furniture, and a matching occupancy map;
- to teleoperate the robot with the keyboard and to provide a SLAM mapping mode;
- to make the robot navigate autonomously with Nav2 and complete a multi-room patrol in closed loop, verified against simulator ground truth;
- to find, explain and fix the navigation problems met on the way; and
- to document the commands and results so that the work can be repeated.

The scope is simulation only. Real sensor behaviour beyond Gaussian noise, motor dynamics, the real Scout Mini firmware, moving people and hardware tests are not covered.

2. Robot Specifications

The first column of the table gives the published figures of the real AgileX Scout Mini as quoted in the project presentation (product sheet as listed by a reseller; other sellers list slightly different values). The second column gives the values of the simplified model that was actually simulated, measured from the SolidWorks meshes and written in `Assem2.SLDASM.urdf`.

Parameter	Real Scout Mini (datasheet)	Simulated model (this project)
Dimensions (L × W × H)	627 × 550 × 252 mm (other sellers: 612 × 580 × 245 mm)	Chassis mesh 672 × 580 × 225 mm; planning footprint 0.66 × 0.70 m
Mass	20 kg (sellers list 23–26 kg)	28 kg (chassis 22 kg + four wheels of 1.5 kg); the CAD mass of 127.8 kg was replaced
Drive	4WD skid-steer, turns in place	4WD skid-steer; Gazebo DiffDrive on joints j1, j2 (left) and j3, j4 (right)
Track (wheel separation)	not used	0.612 m
Wheel size	not listed	Mesh 180 mm diameter; 160 mm (radius 0.08 m) in DiffDrive and collision

Parameter	Real Scout Mini (datasheet)	Simulated model (this project)
Maximum speed	10 km/h (about 2.8 m/s)	Plugin limit 2.5 m/s and 3.5 rad/s; Nav2 cruise 0.6 m/s, smoother cap 0.7 m/s and 1.0 rad/s
Turning radius	0 (spins in place)	0, obtained with wheel friction $\mu = 0.25$
Wheel contact	not listed in the project sources	Cylinder collision shapes, $\mu 1 = \mu 2 = 0.25$
Control input	not listed in the project sources	/cmd_vel (geometry_msgs/Twist) from teleop or Nav2
Battery / payload	not listed in the project sources	Not modelled

Applications

- Research platform for mobile-robot navigation, SLAM and control (the use in this project).
- Indoor inspection and delivery between rooms of a building, as simulated here by driving a patrol through a civil facility.

3. Sensors Used and the Reason of Usage

Three sensors are modelled on the robot and the wheel encoders give odometry. The navigation stack uses the LiDAR, the IMU and the odometry; the camera feeds the first-person view of the ground station.

Sensor	Specification (as simulated)	Reason for use
2D LiDAR (Gazebo `gpu_lidar`, frame `lidar_link`)	720 samples over 360°, range 0.12–16 m, 20 Hz, Gaussian noise $\sigma = 0.01$ m, topic `/scan`; mounted on top of the chassis, scan plane about 0.36 m above the floor	Gives the distance to walls, posts and furniture all around the robot. It is the input of AMCL (localisation on the map), the local costmap (marking and clearing) and of SLAM Toolbox in mapping mode.
IMU (`imu_link`)	Gyroscope and accelerometer, 100 Hz, topic `/imu`, Gaussian noise	Gives the yaw rate accurately. Wheel odometry over-reports rotation on a skid-steer base, so the EKF takes the yaw rate from the IMU.
RGB-D camera (`camera_link`)	640 × 480, 30 Hz, 80° field of view, depth to 10 m; the colour image is bridged as `/camera/image_raw`	Front view shown in the ground station. It is not used by Nav2 in the civil mission.
Wheel odometry (DiffDrive)	Published on `/odom` at 50 Hz, frame odom → base_footprint	Smooth short-term motion estimate; corrected by the IMU in the EKF and by AMCL against the map.

A 2D LiDAR was chosen because the facility is a flat floor and the walls and furniture are taller than the scan plane, so one plane sees every obstacle the robot could hit. It is also the standard input of the ROS 2 localisation and navigation packages.

Wheel odometry is not a separate sensor: the Gazebo DiffDrive plugin integrates the motion of the four driven wheels. On a skid-steer base the wheels slip sideways when turning, so the integrated yaw drifts (about 60–65 % of the commanded yaw was measured before the IMU was fused). The LiDAR and the map correct this through AMCL.

4. SolidWorks Design

The robot was modelled in SolidWorks 2023. The parts are the chassis top (`chassis_top`), the upper deck with its two rails (`up`) and the wheel with its axle (`W`). They were assembled in `Assem2`: one chassis, the upper deck and wheels W1, W2, W5 and W6. The assembly contains a URDF Export Configuration with a global origin and an origin and axis for each of the four wheel joints (j1 to j4), which the SW2URDF exporter reads.

Note: the CAD assembly names the four wheel instances W1, W2, W5 and W6; in the URDF they are the four links w1 to w4 on joints j1 to j4. The chassis mesh is 672 mm along its own X axis, which the 90° yaw of base_footprint turns across the robot, so the body is about 0.58 m long and 0.67 m wide; the 0.66 × 0.70 m Nav2 footprint adds a safety margin around it.

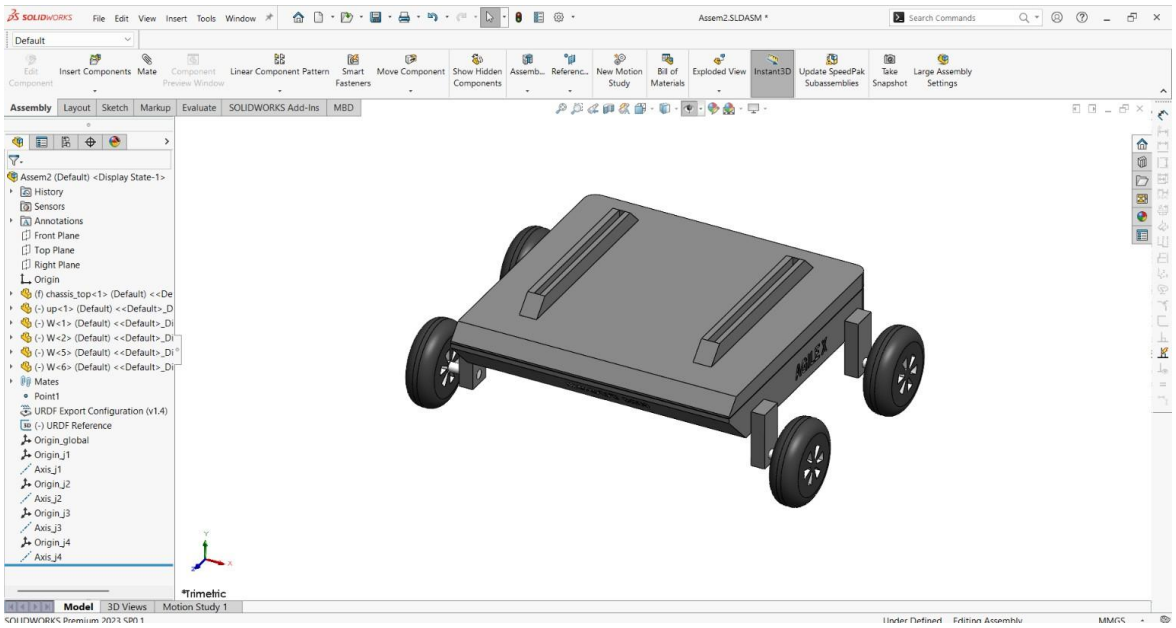


Fig. 1: Assem2 assembly in SolidWorks with the feature tree (chassis_top, up, four wheels, mates, URDF export frames)

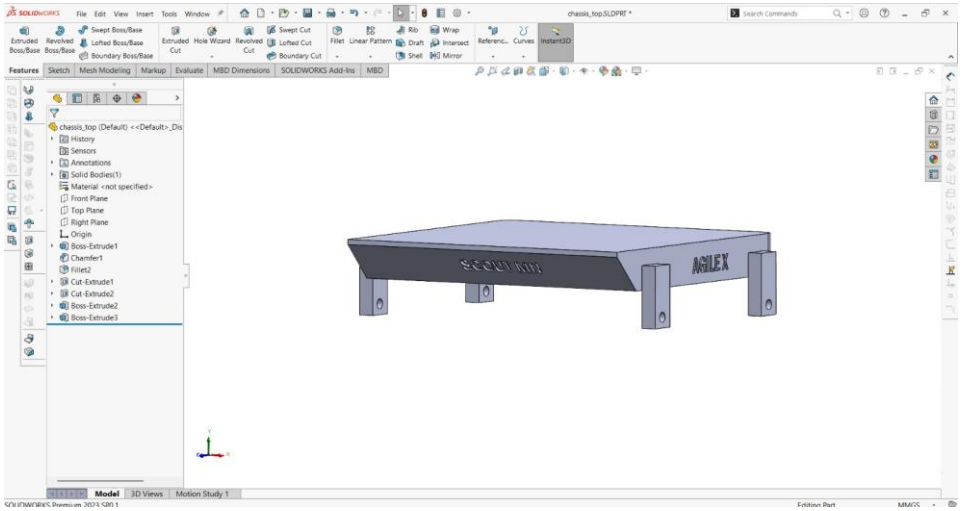


Fig. 2: Chassis part chassis_top

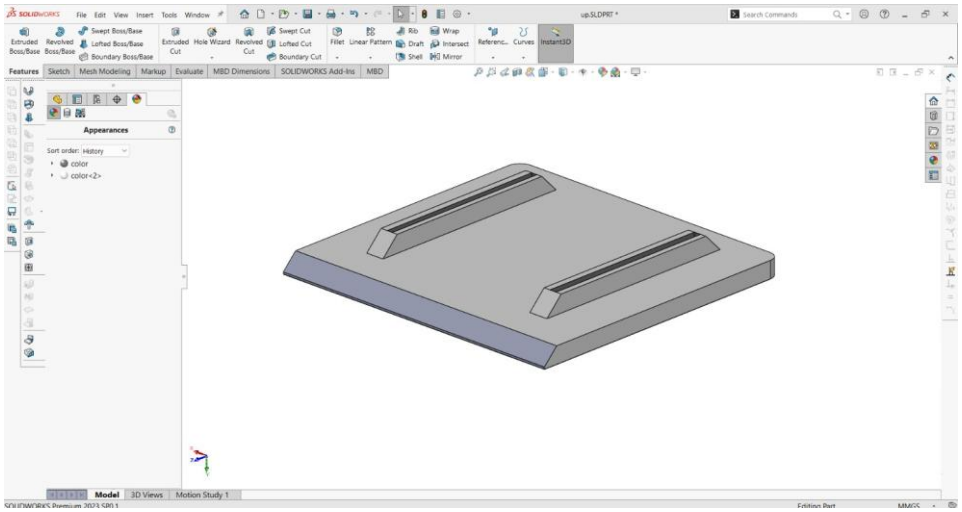


Fig. 3: Upper deck part with two rails

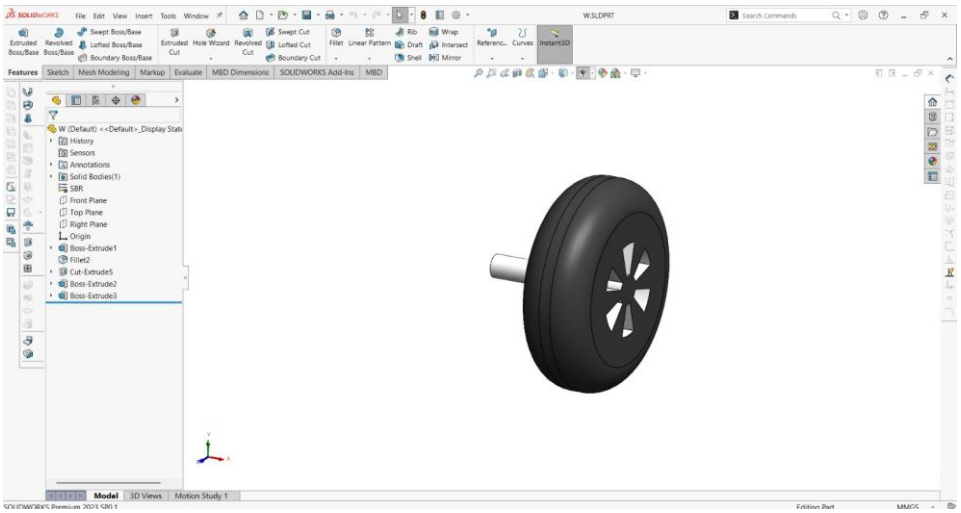


Fig. 4: Wheel part with axle

Dimensions measured from the CAD meshes and used in the simulation: wheel radius 0.08 m, track 0.612 m, chassis mesh about 672 × 580 × 225 mm. The mass given by the CAD (127.8 kg) was not realistic for the robot and was replaced by 22 kg for the chassis plus 1.5 kg per wheel.

5. Conversion to URDF

The Unified Robot Description Format (URDF) is the XML file format ROS uses to describe a robot: its links (rigid parts with visual shape, collision shape, mass and inertia) and joints (how the links are connected and how they move). The assembly was exported with the SolidWorks-to-URDF exporter (configuration v1.4), which produced the URDF, the STL meshes (`base_link.STL`, `w1.STL` to `w4.STL`) and a package description.

The raw export could not be used in Gazebo directly. It was edited into `Assem2.SLDASM.urdf` in the package `assem2_robot`, with these corrections:

Item	Problem in the export	Correction
Frames	The SolidWorks origin sits on top of the chassis, so the robot appeared 0.23 m below the RViz grid and faced sideways	Added a massless `base_footprint` root and a fixed joint to `base_link` with the calibrated offset (−0.10185, 0.16662, 0.2269) and a yaw of 90° so +X is forward (REP-103)
Wheel joints	Wheels exported as fixed or unconstrained joints	Four `continuous` joints j1 to j4 with damping 0.1 and friction 0.1
Wheel axes	Positive velocity drove the robot backwards	Axes calibrated so equal positive wheel speeds drive forward
Mass and inertia	CAD mass 127.8 kg; centre of mass above the chassis, so the robot could tip	Chassis 22 kg with a low centre of mass; wheels 1.5 kg each
Collisions	STL meshes give unstable contact	Cylinders for the wheels (radius 0.08 m, length 0.06 m); chassis keeps its mesh
Wheel friction	μ = 1.0 stopped the robot from turning (skid-steer needs slip)	μ1 = μ2 = 0.25 on all wheels
Drive model	None	One DiffDrive block: left joints j1, j2; right joints j3, j4; separation 0.612 m; radius 0.08 m; 50 Hz odometry
Sensors	No sensors	gpu_lidar on lidar_link, IMU on imu_link and RGB-D camera on camera_link, each on its own fixed joint

The frame tree of the final model is shown below. The EKF publishes `odom → base_footprint`, AMCL publishes `map → odom` and the joint transforms come from `robot_state_publisher`.

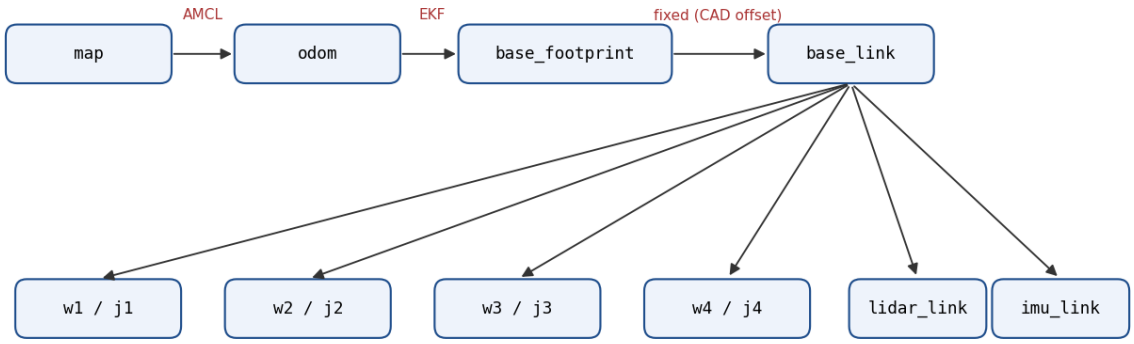


Fig. 5: TF tree of the robot (map → odom → base_footprint → base_link → wheels and sensors)

The DiffDrive block of the URDF, which turns `/cmd_vel` into the four wheel speeds and publishes the odometry:

```
<plugin filename="gz-sim-diff-drive-system" name="gz::sim::systems::DiffDrive">
  <left_joint>j1</left_joint>    <left_joint>j2</left_joint>
  <right_joint>j3</right_joint> <right_joint>j4</right_joint>
  <wheel_separation>0.612</wheel_separation>
  <wheel_radius>0.08</wheel_radius>
  <max_linear_velocity>2.5</max_linear_velocity>
  <max_angular_velocity>3.5</max_angular_velocity>
  <topic>/cmd_vel</topic>    <odom_topic>/odom</odom_topic>
  <frame_id>odom</frame_id> <child_frame_id>base_footprint</child_frame_id>
  <odom_publish_frequency>50</odom_publish_frequency>
</plugin>
```

6. Importing URDF to ROS

6.1 Commands used

The package `assem2_robot` (CMake package with URDF, meshes, worlds, maps, config, launch files and scripts) is built in the workspace `~/agilex` and the model is shown in RViz2 before running Gazebo:

```
cd ~/agilex
source /opt/ros/jazzy/setup.bash
colcon build --symlink-install
source install/setup.bash

# robot model + world + bridge + RViz2 (no SLAM, no Nav2)
./run_civil_sim.sh
# checks
ros2 topic list
ros2 topic echo --once /joint_states
ros2 run tf2_tools view_frames
```

6.2 Failures and what was done

Several problems appeared while moving the exported model into ROS 2 and Gazebo Harmonic. They are listed in the order met, with the cause found and the fix. Dates are the dates of the git commits that fixed them; Aug 2026 means the first import period.

#	Date	Failure	Cause	Fix
1	Aug 2026	ValueError: package name 'assem2.robot' not valid	A period in the package name violates ROS naming rules	Renamed the package to `assem2_robot`
2	Aug 2026	FileNotFoundError for the URDF	Linux file names are case-sensitive	File name made `Assem2.SLDASM.urdf` in the code and in CMakeLists.txt
3	Aug 2026	RViz: could not load the mesh resource	Mesh paths still pointed to the exporter's package name	Mesh URIs set to `package://assem2_robot/meshes/`

#	Date	Failure	Cause	Fix
4	Aug 2026	Wheels missing from the TF tree	Exporter wrote the wheel joints as fixed	Joints changed to continuous
5	Aug 2026	Wheel j3 spun by itself at rest	No axis, no damping on the joint	Axis and damping 0.1 added; all four wheels in one DiffDrive block
6	Aug 2026	Gazebo Classic plugins did not load	ROS 2 Jazzy uses Gazebo Harmonic	Switched to `gz::sim::systems::DiffDrive` and ros_gz_bridge
7	15 Aug 2026	Robot tipped and sat 23 cm below the grid	CAD origin on top of the chassis; high centre of mass	base_footprint added; centre of mass lowered; cylinder wheel collisions
8	17 Aug 2026	Robot drove backwards on a forward command	CAD axes rotated by 90°; wheel axes inverted	Wheel axes and base_footprint yaw calibrated
9	17 Aug 2026	Nodes stalled or published jerky transforms	Two `/clock` publishers (bridge and simulator)	Duplicate clock removed; all nodes use simulated time
10	17 Aug 2026	Drift between Gazebo and RViz during turns	Skid-steer wheel slip in the odometry	robot_localization EKF fusing `/odom` and `/imu`
11	Aug 2026	Duplicate robot in Gazebo	Spawner allowed renaming	Spawn with renaming off; stale processes cleaned
12	25 Oct 2026	Gazebo could not find the meshes and world	Resource path pointed to the package, not its parent share directory	GZ_SIM_RESOURCE_PATH set to the parent share directory in the launchers

6.3 Final result in Gazebo

After the corrections the robot appears correctly in the facility and drives forward on a forward command. The four wheels sit symmetrically, the LiDAR and the camera are mounted on the chassis, and the TF tree is complete. The topics available once the simulation runs are:

```
/clock                  /cmd_vel              /odom
/imu                    /scan                 /joint_states
/camera/image_raw       /camera/camera_info   /odometry/filtered
/robot_description      /tf                   /tf_static
```

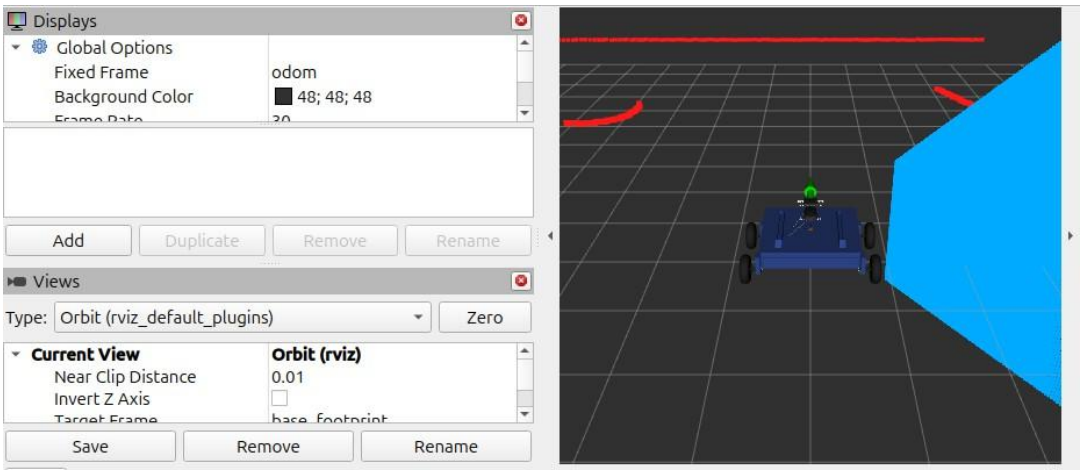


Fig. 6: Robot model with the LiDAR scan (red line) in RViz2

7. Creating the Civil Facility World in Gazebo

The world is a Gazebo Sim (Harmonic) file, `civil_world.sdf`, and its occupancy map is generated by one script, `scripts/generate_civil_world.py`, so that the walls the LiDAR sees in Gazebo and the walls in the Nav2 map are identical. The facility is a 20 m × 16 m building (floor area 320 mZ) with:

- outer walls and partitions 1.2 m high and 0.2 m thick, so every wall is seen by the LiDAR plane;

- a 2.4 m wide central corridor running east–west, with cylindrical posts (diameter 0.3 m) on its centre line at x = ±4.0 m that act as chicanes;
- six rooms opening onto the corridor: Materials Testing Lab, Civil Conference Hall and BIM/CAD Studio on the north side; Geotechnical Survey Lab, Entrance Foyer and Archives Vault on the south side;
- doorways 1.2 m wide (the Conference Hall door 1.4 m wide) at x = −5.9, 0 and +5.9 m, and an open 4 m atrium to the foyer; the robot is 0.67 m wide, so there is about 0.26 m of slack per side;
- 0.45 m square structural columns and furniture in every room (workbenches, a compression press, a conference table, a workstation island, shelving racks and a soil-testing rig);
- seven named stations S1 to S7 used as navigation goals: Foyer (home), Corridor Hub, Materials Lab, Conference Hall, CAD Studio, Archives Vault and Geotechnical Lab.

The robot is spawned at the world origin, the corridor hub, facing +Y (yaw 90°); the mission seeds the AMCL start pose at this point.

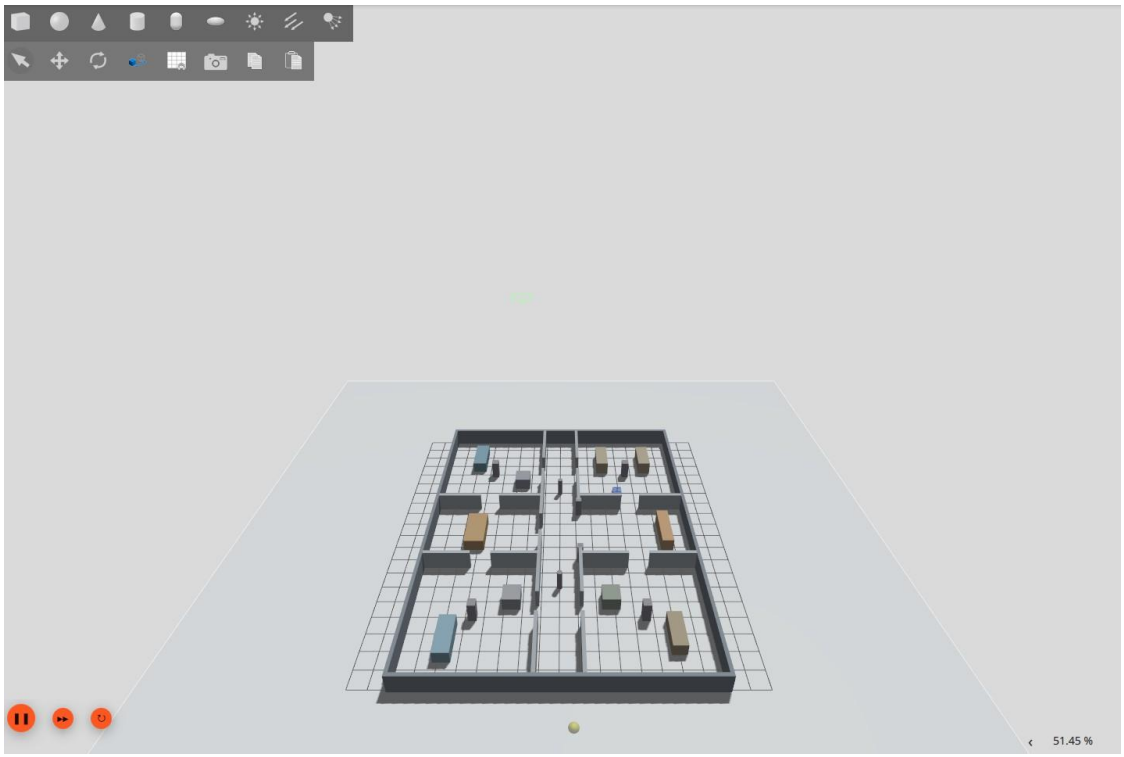


Fig. 7: The civil facility in the Gazebo GUI (generated world, civil_world.sdf)

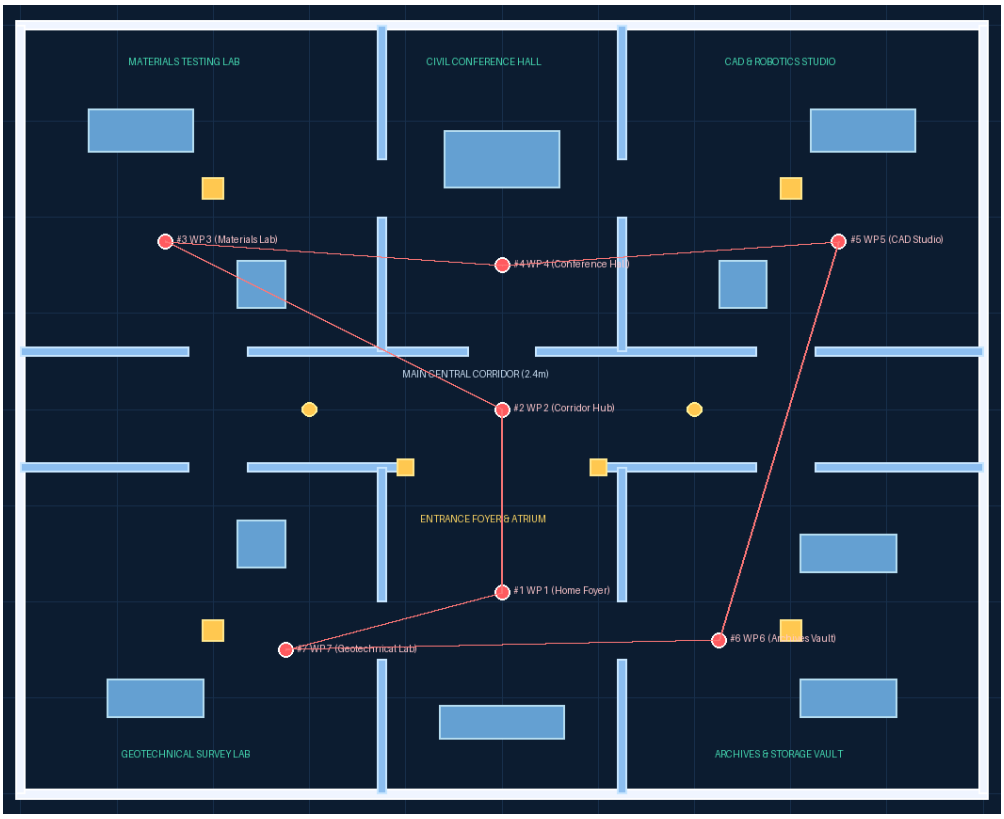


Fig. 8: Blueprint of the facility drawn by the generator script with room names, doors and columns

Commands

```
# regenerate the world and the map (overwrites civil_world.sdf and civil_map.*)
cd ~/agilex/src/assem2_robot/scripts
python3 generate_civil_world.py

# world + robot + bridge + RViz2, no SLAM, no Nav2
cd ~/agilex && ./run_civil_sim.sh
```

8. Creating the Map

Two ways of producing the occupancy map are available in the project.

Generated map (used for navigation). The generator script draws the walls, columns and furniture into a grid with 5 cm cells and writes `civil_map.pgm` and `civil_map.yaml`. Free floor is value 254, obstacles 0 and the exterior is unknown (205). For a world point (x, y) the cell is u = round((x − X0)/0.05), v = round((y − Y0)/0.05) with the origin (X0, Y0) = (−11.0, −9.0) m. The grid is 440 × 360 cells, which is 22 m × 18 m.

```
image: civil_map.pgm
mode: trinary
resolution: 0.0500
origin: [-11.000, -9.000, 0.000000]
negate: 0
occupied_thresh: 0.65
free_thresh: 0.25
```

SLAM mode. The `slam_toolbox` package builds a map online while the robot is driven by hand. Each LiDAR scan is matched against the map built so far to correct the odometry; the corrected poses form a pose graph that is optimised (Ceres solver) when the robot revisits a place, and every scan updates a 5 cm occupancy grid by ray tracing. The launcher and the saver are:

```
./run_civil_mapping.sh              # Gazebo + RViz2 + EKF + slam_toolbox
./run_teleop.sh                     # second terminal: drive slowly through the rooms
./save_civil_map.sh my_map          # writes maps/my_map.yaml and my_map.pgm
```

The SLAM parameters are in `slam_toolbox.yaml`: a new graph node every 0.15 m or 0.15 rad, map update every 0.5 s, LiDAR range 0.12–16 m. Passing no name to `save_civil_map.sh` writes `civil_map`, which overwrites the map used by Nav2, so a new name must always be given. The SLAM mode and the teleoperation scripts were run when they were written but were **not re-run** after the later navigation fixes, so they are listed as unverified in the command sheet.

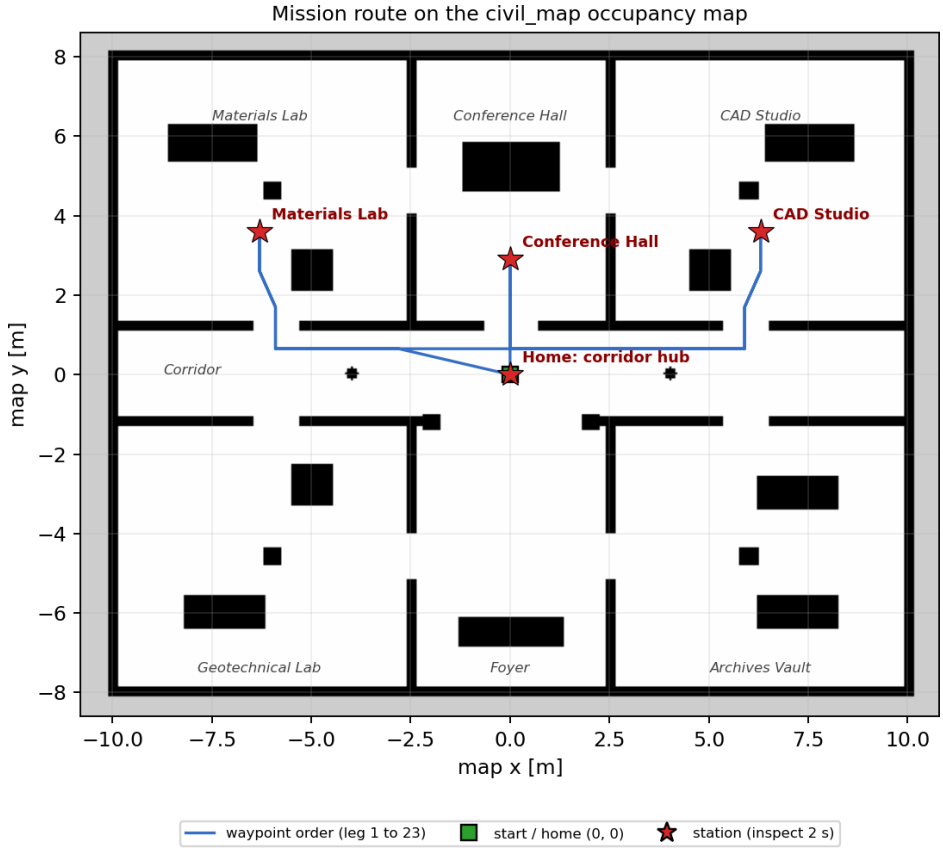


Fig. 9: The saved occupancy map (black = occupied, white = free, grey = unknown) with the 23-leg mission route drawn on it; the blue line joins the waypoints in order and is not the measured path

9. Teleoperating the Robot Through the Facility

In teleoperation the robot is driven by a person. The script `civil_teleop.py` reads the keyboard in raw mode and streams a velocity command (linear v, angular ω) as `geometry_msgs/Twist` at 25 Hz on `/cmd_vel`. `ros_gz_bridge` passes it to Gazebo, where the DiffDrive system converts it into the wheel speeds. For the 4WD skid-steer base treated as an equivalent differential drive with track b = 0.612 m and wheel radius r = 0.08 m:

```
w_R = (v + w*b/2) / r        w_L = (v - w*b/2) / r        b = 0.612 m, r = 0.08 m
```

For example, v = 0.5 m/s and ω = 0 gives all four wheels 6.25 rad/s. The same plugin integrates the wheel odometry on `/odom`. The velocity is ramped with an S-curve so the skid-steer wheels do not slip and the LiDAR plane does not pitch. Cruise control is the default mode (press to drive, Space to stop); a momentary hold-to-drive mode can be toggled with t.

Commands

```
# Terminal 1: Gazebo + robot + RViz2
cd ~/agilex && ./run_civil_sim.sh
# Terminal 2: keyboard teleop
cd ~/agilex && ./run_teleop.sh
# check what is being sent
ros2 topic echo /cmd_vel
ros2 topic echo /odom --once
```

Keys: W/S forward and reverse, A/D turn, Q/E and Z/C arcs forward and reverse, Space (or k, x) emergency stop, + and − change the speed, r resets to the recommended mapping speeds. Arrow keys and the classic ROS keys i, j, k, l, u, o, m, '.' also work.

10. Autonomous Navigation

For autonomous navigation the saved map is loaded and the robot is given goals. Nav2 localises the robot on the map with AMCL, plans a route on the global costmap, follows it with the controller and avoids new obstacles seen by the LiDAR. Goals can be given in three ways: a station button or a click on the floor map in the **ground station**, the 2D Goal Pose tool of RViz2, or the script `civil_nav2_mission`.

Commands

```
# one command: clean simulation, Nav2, window tiling, screen recording and mission
./run_patrol_and_record.sh

# or in two terminals
./run_civil_nav.sh                                  # Terminal 1: Gazebo, Nav2, RViz2, ground station
ros2 run assem2_robot civil_nav2_mission            # Terminal 2: after 'Managed nodes are active'

# a single goal from the command line
ros2 action send_goal /navigate_to_pose nav2_msgs/action/NavigateToPose \
  "{pose: {header: {frame_id: map},
           pose: {position: {x: 0.0, y: 2.9},
                  orientation: {z: 0.7071, w: 0.7071}}}}"
```

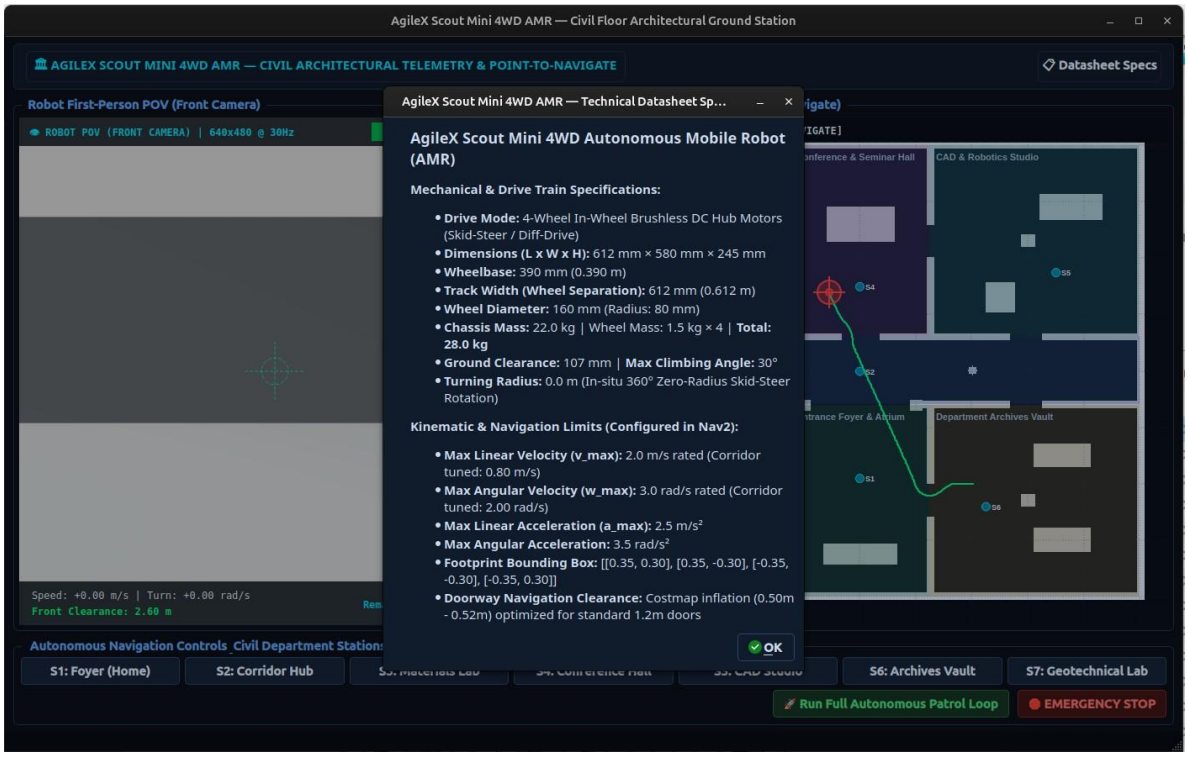


Fig. 10: Ground station (PyQt5) with the robot's first-person view, the floor map with a planned route, station buttons S1–S7, an emergency stop, and the datasheet window

The mission

`civil_nav2_mission` sends 23 goals one after another. Each goal is a real Nav2 `NavigateToPose` action, so every leg uses AMCL, the planner, the costmaps and the controller; nothing is played back open-loop. The route goes from the corridor hub to the Conference Hall, along the east lane to the CAD Studio, back along the corridor to the Materials Lab, and home. Door-alignment waypoints make the robot cross each 1.2 m door straight, and the goal heading is set to the direction of travel so the robot never has to spin on arrival. Four stations (Conference Hall, CAD Studio, Materials Lab and the final home position) stop for 2 s of simulated inspection.

An arrival counts as passed only when the Nav2 result is *succeeded* and the robot is within 0.45 m of the goal both by AMCL and by the Gazebo ground-truth pose (`gz model -p`), so a localisation error cannot hide a wrong arrival. A failed leg is retried up to three times with the costmaps cleared.

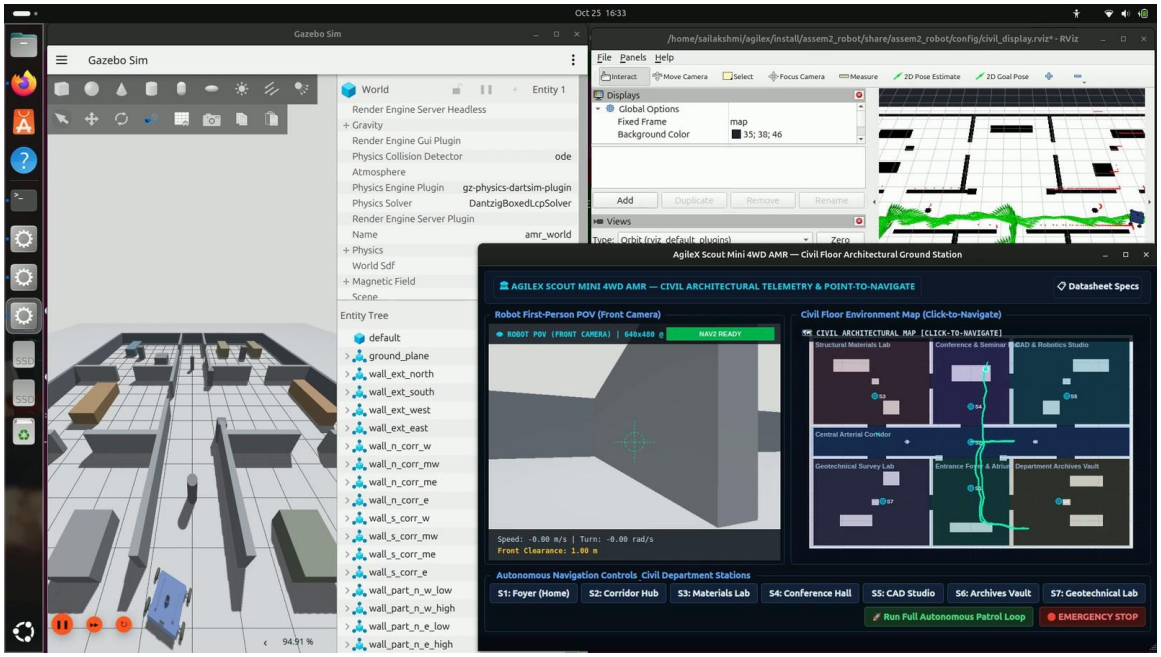


Fig. 11: One autonomous run: Gazebo (left), RViz2 (top right) and the ground station with the robot's route trace (bottom right), recorded at 24 frames per second

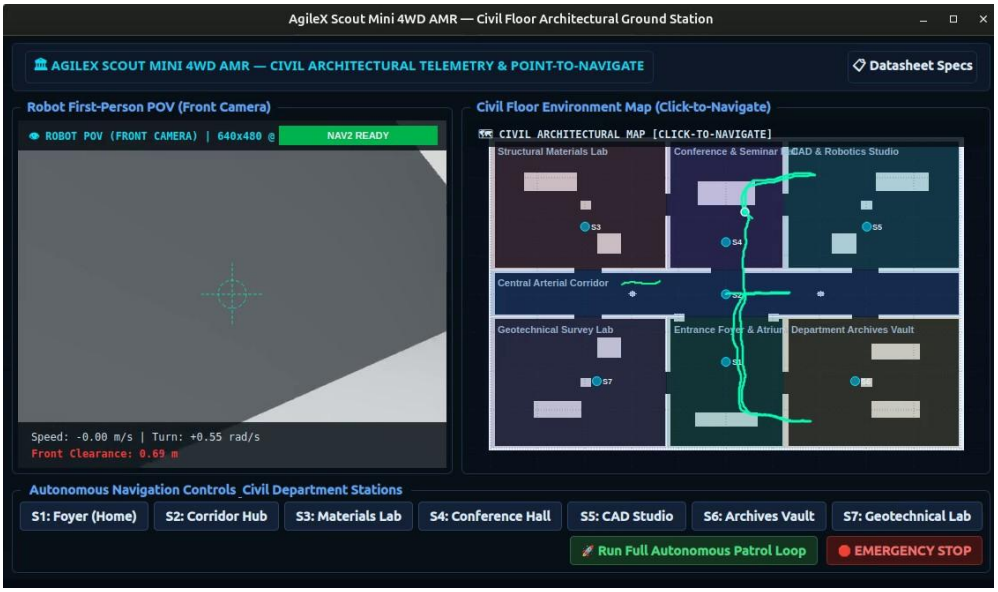


Fig. 12: Ground station late in the run: the cyan trace shows the path already driven through the rooms and the corridor

Results

The table lists the recorded run of the closed-loop mission (report file recordings/agilex_civil_patrol_demo_report.txt). Yaw error is the arrival heading error; it is only a pass criterion (40°) at the stations.

Leg	Waypoint	AMCL error [m]	Truth error [m]	Yaw error [°]
1	Conference Hall door	0.18	0.22	4
2	Station: Conference Hall	0.17	0.21	8
3	Conference Hall door (out)	0.17	0.25	14
4	Corridor hub	0.18	0.27	14
5	East lane (past post)	0.18	0.31	18
6	East lane (CAD door)	0.20	0.26	17
7	CAD Studio door (through)	0.19	0.33	5
8	CAD Studio (clear of rig)	0.19	0.19	7
9	Station: CAD Studio	0.20	0.27	7
10	CAD Studio (clear of rig)	0.17	0.22	11
11	CAD Studio door (through)	0.17	0.23	1
12	Corridor (CAD door)	0.20	0.30	3
13	Corridor hub (west bound)	0.19	0.25	18
14	West lane (past post)	0.18	0.26	18
15	West lane (Materials door)	0.17	0.24	20
16	Materials Lab door (through)	0.18	0.21	1
17	Materials Lab (clear of press)	0.18	0.21	2
18	Station: Materials Lab	0.18	0.21	4
19	Materials Lab (clear of press)	0.20	0.31	2
20	Materials Lab door (through)	0.18	0.28	6
21	Corridor (Materials door)	0.18	0.27	2
22	West lane (past post)	0.19	0.24	19
23	Home: corridor hub	0.17	0.20	11

Result of the final recorded run (reached only after the failed runs described in Section 11): 23 of 23 legs reached with no retries. The mean AMCL error at the goals was 0.18 m (maximum 0.20 m), the mean error against Gazebo ground truth was 0.25 m (0.19–0.33 m) against the 0.45 m limit, and the mean heading error was 9° (maximum 20°). The scripted run, including the start-up of the simulation, takes about four minutes.

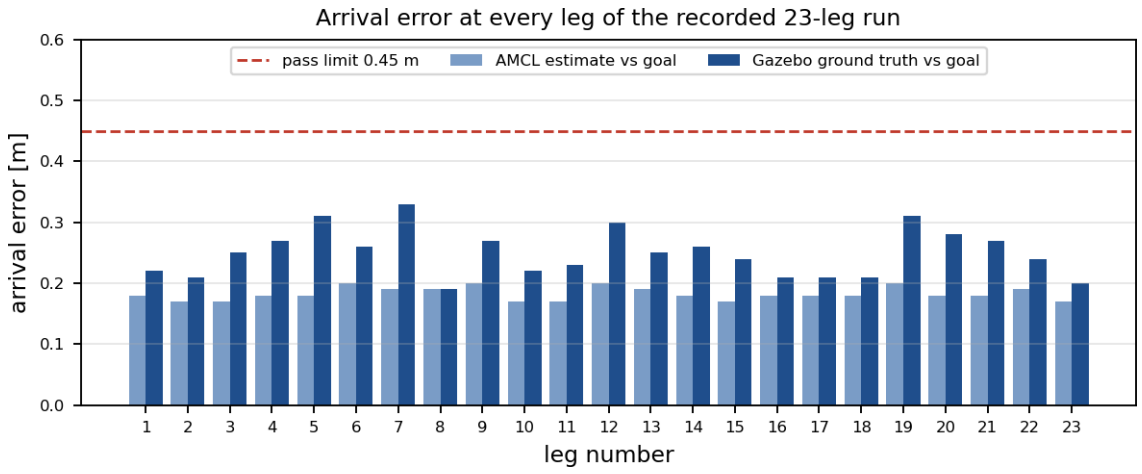


Fig. 13: Arrival error at each leg, from the recorded run

This is **one** successful recorded run of the final configuration. It was preceded by many failed runs: each configuration tried earlier failed at a different leg or behaviour (for example turning, a collision with the outer wall, wedging on door jambs, AMCL losing the robot, or a planner failure), and each failure led to one of the fixes in Section 11. The exact number of attempts was not recorded. The final configuration has not yet been repeated many times, so the result shows that the system can work, not how often.

Failed runs before the final run

The 23/23 run is the last of many attempts. The table lists the failed attempts that are documented in the project's issues log (`NAVIGATION_ISSUES_LOG.md`) with what was observed and measured. The order and dates of the individual attempts, and how many times each was repeated, were not recorded, so none are claimed; the last column points to the cause and fix in Section 11.

Failed run	Setup at the time	What was observed	Outcome	Section 11
F1	Open-loop patrol script (`civil_patrol_mission`): timed velocity commands	Worked only while nothing drifted; no localisation, planning or obstacle feedback	Not a valid autonomous mission; replaced	#7
F2	First Nav2 run: 1.55 m/s, collision detection off, rotation 2.5 rad/s	Robot drove about 8 m through the outer wall; map y reached 8.9 m with the wall at 8.0 m	Failed (collision)	#8
F3	Turning test with wheel $\mu = 1.0$	Commanded rotation gave about 0° of yaw; arcs turned about 17° of 137°	Failed (could not turn)	#1
F4	Turn in place with the original acceleration limits	Constant 0.10 rad/s command but no motion	Failed (deadlock)	#2
F5	Costmap and AMCL with default LiDAR minimum range	“Collision ahead” on open floor; AMCL scan matching corrupted by hits on the robot's own front	Failed (false obstacles)	#4
F6	NavFn global planner in narrow passages	“Failed to create a plan from potential”	Failed (no plan)	#9
F7	Waypoint at (4.5, 0.0) in the corridor	Planner failed: a post stands on the corridor centre line at x = 4.0 m	Failed (goal blocked)	#10
F8	Goal heading 180° on arrival	Spin never converged; the controller flipped ±0.6 rad/s every 1.5 s (heading about 270° from arrival heading)	Failed (no arrival)	#11
F9	Footprint 0.58 m wide through doors and past furniture	Robot wedged on door jambs and furniture; AMCL pose ran 0.6–1.7 m ahead of the true pose	Failed (stuck)	#3
F10	AMCL with tight motion noise and recovery off	AMCL ended 4 m from the true pose in the CAD Studio (similar rooms)	Failed (lost)	#5
F11	Door crossing with 0.26 m slack per side	Front corner clipped the door jamb when leaving a room	Failed (contact)	#12
F12	Arrival checked only against AMCL	Legs reported OK were wrong by up to 1.4 m	False pass; ground-truth check added	#6
F13	Runs stopped on timeout	Goals were not cancelled; /cmd_vel stayed busy while idle and contaminated the next test	Failed (stale goal)	#13
Final	Final configuration, recorded run	23/23 legs reached, but about 590 “collision ahead” warnings caused 2–3 s pauses entering the CAD Studio and Materials Lab	Passed, with pauses (open issue)	Sec. 16

11. Navigation Problems Found and Solved

Sixteen problems were found by running the mission (the failed runs F1 to F13 in Section 10), reading the logs and comparing the robot's own pose estimate with the Gazebo ground truth. Each was traced to its cause and fixed. They fall into three groups: robot model and sensing, planning and control, and tooling.

11.1 Robot model, localisation and sensing

#	Problem	Root cause	Fix
1	Robot could not turn: commanded rotation gave about 0° of yaw	Wheel friction μ = 1.0; the physics engine used one μ, so the lower μ2 was ignored	μ1 = μ2 = 0.25 in the URDF; in-place turns work
2	Turn-in-place deadlock: constant 0.10 rad/s command, no motion	Angular acceleration limit applied from measured velocity (0), so static friction was never broken	max_angular_accel 8.0 in the controller and 6.0 in the smoother
3	Robot wedged on door jambs and furniture; AMCL pose ran 0.6–1.7 m ahead of truth	Nav2 footprint 0.58 m wide, real robot about 0.67 m (track 0.612 m plus wheel width)	Footprint 0.66 × 0.70 m, inflation radius 0.55 m, scale 4.0
4	"Collision ahead" in open floor; AMCL matching corrupted	LiDAR hits on the robot's own front (0.25–0.5 m) marked as obstacles	Obstacle min range, raytrace min range and AMCL laser_min_range set to 0.55 m
5	AMCL lost the robot in similar rooms (4 m off in the CAD Studio)	Motion noise too tight for skid-steer slip; recovery disabled	alpha1 0.30, others 0.10; 1000–4000 particles; recovery alphas 0.1 / 0.001
6	Wrong "OK" results: legs passed that were wrong by 1.4 m	Arrival checked only against AMCL's own pose	Every arrival also checked against Gazebo ground truth

11.2 Planning and control

#	Problem	Root cause	Fix
7	"Autonomous patrol" was not autonomous	The first patrol script replayed timed `/cmd_vel` commands with no localisation or feedback	New mission script: every leg is a Nav2 NavigateToPose goal
8	Robot drove about 8 m through the outer wall	Nav2 ran at 1.55 m/s with collision detection off and 2.5 rad/s rotation	0.6 m/s cruise, collision detection on, rotation 0.6 rad/s
9	"Failed to create a plan" in narrow passages	NavFn planner	Switched to Smac Planner 2D
10	Planner failed for a goal at (4.5, 0.0)	Posts stand on the corridor centre line at x = ±4.0 m	Route moved to the north lane (y = 0.65 m)
11	Spin on arrival never converged	Goal heading differed by about 270° from the arrival heading	Goal heading = direction of travel; yaw checked only at stations
12	Corner clipped the door jamb on room exit	Only 0.26 m of slack per side	Doors crossed on the true centre line x = ±5.9 m; lookahead 0.25–0.60 m

11.3 Tooling and verification

#	Problem	Root cause	Fix
13	Leftover goals kept /cmd_vel busy	Timed-out runs never cancelled their goal	The mission cancels its active goal on exit
14	Stale ground-station windows piled up	The GUI is a python3 process, so killall by name missed it	Launcher and record script kill it by PID
15	Recording showed the terminal, not the simulation	No window management	`arrange_windows.py` tiles Gazebo, RViz2 and the ground station
16	Video dropped frames and was large (40 MB)	Fixed-rate loop and mp4v codec	Frames repeated to keep wall-clock timing; H.264, 11 MB

A related fix concerned the ground station: the robot marker was taken from odometry (odom frame) while goals and routes are in the map frame, about 89° apart, so the marker sat in the wrong room. The map→odom transform is now looked up with TF before drawing.

12. Algorithms Used (Smac 2D, AMCL, EKF, RPP, BT …)

Algorithm	Package / file	Purpose	Why this one
EKF (extended Kalman filter)	robot_localization, ekf.yaml	Fuses wheel linear velocity and the IMU yaw rate at 20 Hz in 2D mode; publishes /odometry/filtered and odom → base_footprint	Wheel yaw drifts on a skid-steer base; the gyro does not
AMCL (particle filter)	nav2_amcl, nav2_params.yaml	1000–4000 particles matched to the map with the likelihood-field model; gives map → odom	Corrects the drift of the odometry against the map; needs a start pose (0, 0)
SLAM Toolbox (graph SLAM)	slam_toolbox.yaml	Scan matching, pose graph and loop closure (Ceres) to build a map while driving by hand	Standard 2D LiDAR SLAM; used only for mapping, not for the mission
Costmaps	nav2_costmap_2d	Global: static map + inflation (1 Hz). Local: 4 × 4 m rolling window with LiDAR marking and clearing (5 Hz). Footprint 0.66 × 0.70 m, inflation 0.55 m	Keeps the body off the walls; sees new obstacles
Smac Planner 2D (A*)	nav2_smac_planner	Cheapest route on the grid, f(n) = g(n) + h(n), at most 2 s; a simple smoother removes kinks	Optimal on the grid; replaced NavFn which failed in narrow gaps
Rotation shim + regulated pure pursuit (RPP)	nav2_rotation_shim_controller, nav2_regulated_pure_pursuit_controller	Rotates towards the path if the heading error exceeds 0.52 rad, then follows a lookahead point (0.25–0.60 m) at up to 0.6 m/s, slowing near obstacles and in sharp turns	Smooth, predictable on a skid-steer base; curvature κ = 2y/LZ
Behaviour tree (BT Navigator)	nav2_bt_navigator	Runs navigate-to-pose with replanning; on failure clears costmaps and runs spin, backup, drive-on-heading, wait	Gives recovery without manual help
Velocity smoother (and collision monitor)	nav2_velocity_smoother, nav2_collision_monitor	The velocity smoother limits speed to 0.7 m/s and 1.0 rad/s and acceleration to 1.0 m/sZ and 6.0 rad/sZ. The collision monitor is launched, but its footprint-approach polygon is disabled in nav2_params.yaml, so obstacle stopping relies on the RPP collision check and the costmaps	Smooth commands between the controller and the wheels

The goal is accepted as reached when the robot is within 0.20 m and 0.35 rad of the requested pose; the mission script then applies its own 0.45 m check against the Gazebo ground truth.

Key parameters from the configuration files:

```
planner:      nav2_smac_planner::SmacPlanner2D         # A*, 1 Hz replanning
controller:   nav2_rotation_shim_controller -> RegulatedPurePursuit
              desired_linear_vel 0.60    lookahead 0.25..0.60 m
 goal check:   xy_goal_tolerance 0.20     yaw_goal_tolerance 0.35
costmaps:     resolution 0.05  inflation_radius 0.55  footprint 0.66 x 0.70 m
amcl:         min_particles 1000  max_particles 4000  laser_min_range 0.55
ekf:          frequency 20  two_d_mode  odom0: vx     imu0: yaw rate
```

13. System Architecture

Gazebo Harmonic runs the civil world and the robot with its DiffDrive system (wheel odometry), LiDAR, IMU and camera. `ros_gz_bridge` converts between Gazebo topics and ROS 2 topics (`/clock`, `/cmd_vel`, `/odom`, `/imu`, `/scan`, `/camera`). `robot_state_publisher` builds the TF tree from the URDF. The EKF fuses `/odom` and `/imu` into `/odometry/filtered`. For navigation, `map_server` and AMCL localise the robot on the map, and the Nav2 servers (BT navigator, planner, controller, costmaps, smoother) produce `/cmd_vel`, which returns to the DiffDrive plugin through the bridge. RViz2 and the ground station display the map, the plan and the robot and send goals.

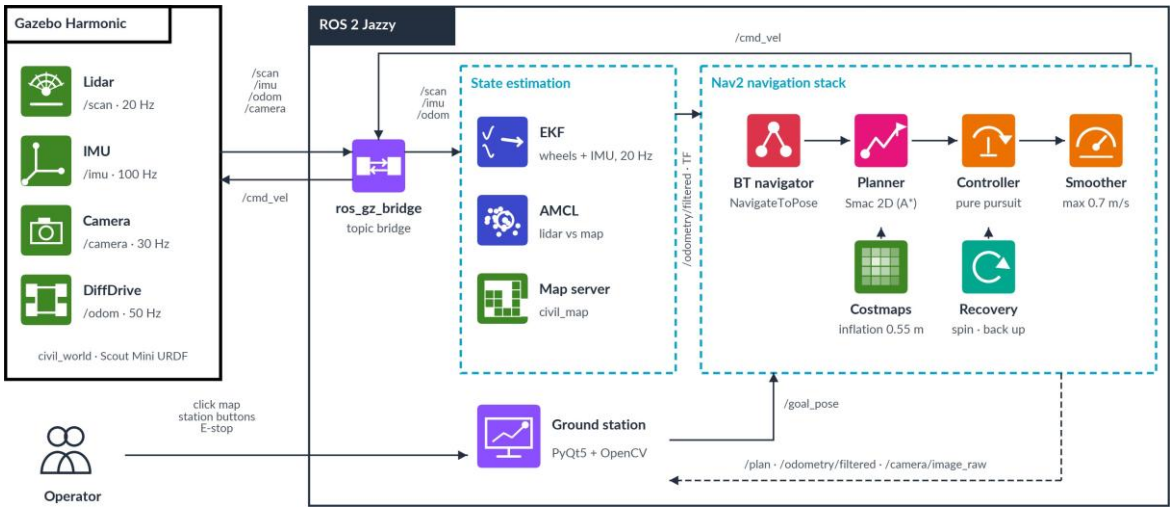


Fig. 14: ROS 2 node and topic architecture of the project (package assem2_robot)

Topic	Type	Publisher → subscriber
`/cmd_vel`	geometry_msgs/Twist	Teleop or Nav2 velocity smoother → bridge → Gazebo DiffDrive
`/scan`	sensor_msgs/LaserScan (lidar_link)	Gazebo LiDAR → AMCL, costmaps, RViz2 (SLAM in mapping mode)
`/imu`	sensor_msgs/Imu	Gazebo IMU → EKF
`/odom`	nav_msgs/Odometry	Gazebo DiffDrive → EKF
`/odometry/filtered`	nav_msgs/Odometry	EKF → Nav2 controller and BT navigator, ground station
`/tf`	tf2_msgs/TFMessage	odom → base_footprint (EKF); map → odom (AMCL or SLAM); wheels and sensors (robot_state_publisher)
`/map`	nav_msgs/OccupancyGrid	map_server (navigation) or slam_toolbox (mapping) → AMCL, global costmap, RViz2
`/camera/image_raw`	sensor_msgs/Image	Gazebo camera → ground station
`/clock`	rosgraph_msgs/Clock	Gazebo → every node (simulated time)

Which launcher starts what: `run_civil_sim.sh` starts Gazebo, the bridge, robot_state_publisher, the EKF and RViz2; `run_civil_mapping.sh` adds slam_toolbox; `run_civil_nav.sh` adds the Nav2 stack, map_server, AMCL and the ground station; `run_patrol_and_record.sh` adds a screen recorder and the mission script. All use ROS_DOMAIN_ID 19 on localhost only.

14. What Happens When the Code Runs (Flowchart)

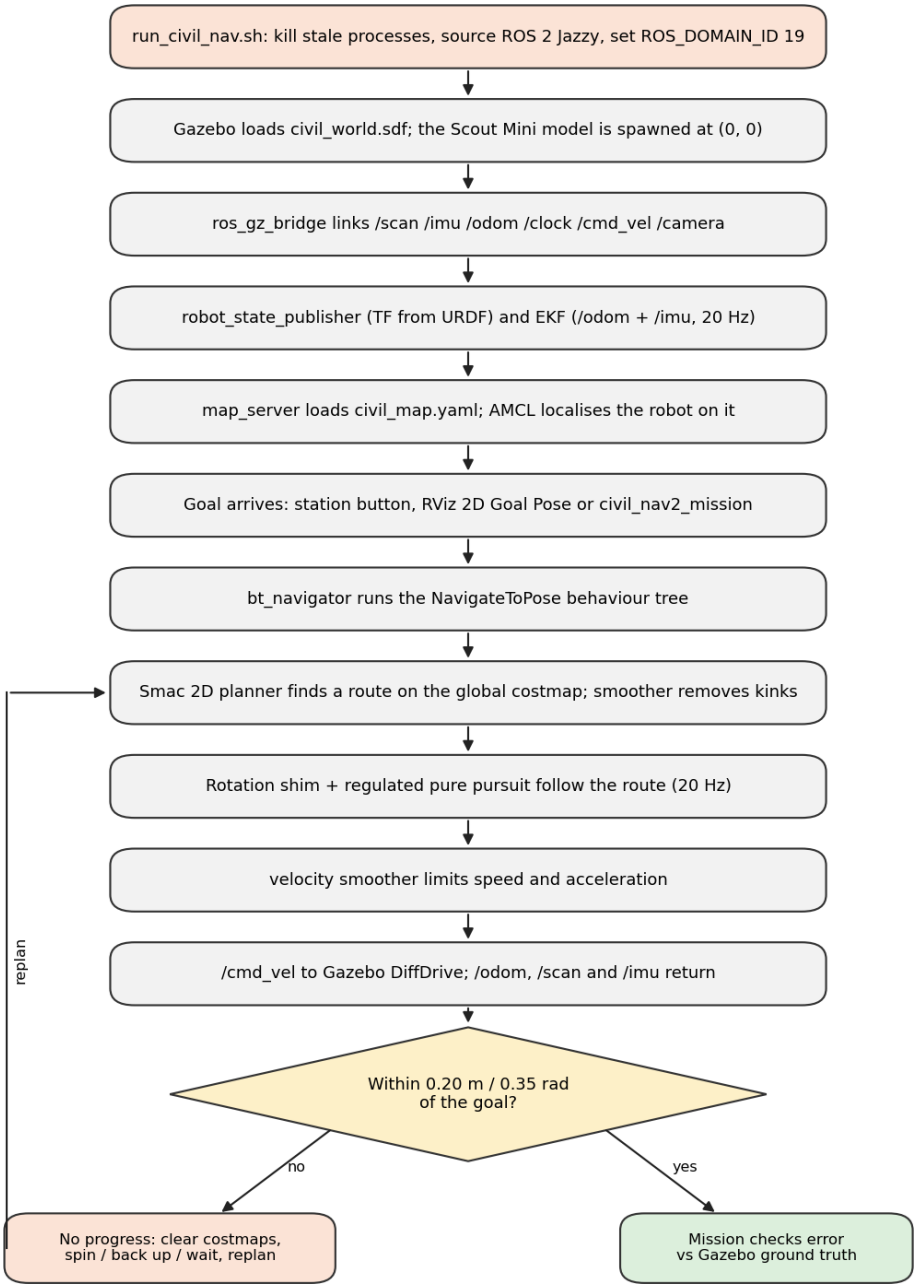


Fig. 15: Flowchart of the system from launch to goal reached

In words: the launcher cleans up old processes, sets up ROS 2 and Gazebo; Gazebo loads the civil world and spawns the robot; the bridge, robot_state_publisher and the EKF make the simulation visible to ROS 2. The map server and AMCL localise the robot on the saved map. A goal starts the behaviour tree, which asks the Smac planner for a route and the controller for velocity commands. The commands pass the velocity smoother and go back to Gazebo, which returns new odometry and scans. This repeats until the robot is within the goal tolerance; if progress stops, the recovery behaviours run and the route is replanned. In the mission script each arrival is then verified against the Gazebo ground truth before the next leg starts.

15. How to Set It Up

15.1 Requirements

- Ubuntu 24.04 with ROS 2 Jazzy and Gazebo Harmonic (the project was built and run on this combination).
- ROS packages: `ros-jazzy-ros-gz`, `ros-jazzy-navigation2`, `ros-jazzy-nav2-bringup`, `ros-jazzy-slam-toolbox`, `ros-jazzy-robot-localization` and `python3-colcon-common-extensions`.
- Python 3 with PyQt5 and OpenCV for the ground station; an X11 desktop at 1920 × 1080 for the window tiling and screen recording.

15.2 Installation and build

```
sudo apt install ros-jazzy-desktop ros-jazzy-ros-gz ros-jazzy-navigation2 \
     ros-jazzy-nav2-bringup ros-jazzy-slam-toolbox ros-jazzy-robot-localization \
     python3-colcon-common-extensions
git clone https://github.com/Saibts/AgileX_ScoutMini_JAZZY.git ~/agilex
cd ~/agilex
source /opt/ros/jazzy/setup.bash
colcon build --symlink-install
source install/setup.bash
```

15.3 Running

```
./run_civil_sim.sh              # 1. simulation only (+ ./run_teleop.sh in a second terminal)
./run_civil_mapping.sh          # 2. SLAM mapping (+ ./save_civil_map.sh my_map)
./run_civil_nav.sh              # 3. Nav2 + ground station
ros2 run assem2_robot civil_nav2_mission    # ... then the mission, in a 2nd terminal
./run_patrol_and_record.sh      # 4. clean run of the whole mission with a recording

# clean stop if a window hangs after Ctrl+C
killall -9 gz ruby parameter_bridge rviz2 robot_state_publisher ekf_node
```

Always start the navigation with a fresh simulation: the mission seeds AMCL at (0, 0), so the robot must be at the world origin. To run it again, restart the simulation first.

15.4 Code layout

Path (~/agilex/src/assem2_robot/)	Content
`urdf/Assem2.SLDASM.urdf`	Robot model: links, joints, sensors, DiffDrive plugin, wheel friction
`meshes/`	STL meshes of the chassis and wheels from SolidWorks
`worlds/civil_world.sdf`, `scripts/generate_civil_world.py`	Civil facility world and the script that generates it with its map
`maps/civil_map.yaml`, `civil_map.pgm`	Occupancy map used by Nav2
`launch/civil_simulation.launch.py`	Master launch file with use_slam, use_nav and use_gui options
`config/nav2_params.yaml`, `ekf.yaml`, `slam_toolbox.yaml`	Nav2, EKF and SLAM parameters
`scripts/civil_nav2_mission.py`	Closed-loop 23-leg mission with ground-truth verification
`scripts/civil_interactive_nav.py`	Ground station (click-to-navigate, live view)
`scripts/civil_teleop.py`	Keyboard teleoperation
`~/agilex/run_*.sh`, `~/agilex/scripts/arrange_windows.py`, `~/agilex/scripts/record_screen.py`	Launchers, window tiling and recording

16. Future Enhancements

- **Repeat the mission** many times and over several layouts so that the results are statistical rather than one recorded run; a repeatability check is the first open item.
- **Brief pauses entering the CAD Studio and Materials Lab:** about 590 “collision ahead” warnings were logged in the last runs, each a 2–3 s stop where the robot's front corner swings within about 0.1 m of the rig or press. Moving the furniture, using a smaller planning footprint with a margin elsewhere, or adding a dedicated approach waypoint should remove them.
- **Calibrate the odometry:** skid-steer slip gives roughly 60–65 % of the commanded yaw; the DiffDrive wheel separation has not been calibrated to the effective track.
- **Do not assume the start pose:** the mission seeds AMCL at (0, 0) and the waypoints are hard-coded to this world; global localisation and a waypoint set generated from the map would remove both limits.
- **Smoother motion:** use NavigateThroughPoses so the robot does not stop at every waypoint.
- **Monitor localisation without ground truth:** on a real robot only the AMCL covariance is available, so the mission should monitor it and re-localise automatically.
- **Re-test the other modes** (SLAM mapping, teleoperation and the 10 m obstacle arena) after the navigation parameter changes, and add moving obstacles and people.
- **Run the same software on the physical Scout Mini**, using the depth camera in the costmap as well as the LiDAR.

17. Conclusion

A SolidWorks model of the AgileX Scout Mini was converted to URDF, corrected for ROS 2 and Gazebo (frames, wheel axes, inertia, friction, drive model) and fitted with a 2D LiDAR, an IMU and a camera. A 20 m × 16 m civil facility and its occupancy map were generated for Gazebo Harmonic. With an EKF fusing wheel odometry and the IMU, AMCL on the map, the Smac 2D A* planner and the rotation-shim regulated pure pursuit controller, the robot completed a closed-loop mission of 23 Nav2 goals through three rooms and back to the corridor hub, with no retries and a worst arrival error of 0.33 m against Gazebo ground truth. This was reached only after many failed runs.

The main difficulties were the wheel friction and footprint, the LiDAR seeing the robot itself, AMCL losing the robot under skid-steer slip, and goal headings that forced spinning on arrival; each was traced to its cause and fixed, and verifying every arrival against the simulator's ground truth exposed errors that the robot's own estimate had hidden. The limits are stated plainly: the success is one recorded run in simulation that followed many failed attempts, the robot still pauses briefly when entering two rooms, the SLAM and teleoperation modes were not re-tested after the final changes, and nothing was tested on hardware.